

MOVING AVERAGES

LAB 1

SECTION:

H

SUBMITTED BY:

Simon Samuel

SUBMISSION DATE:

11/07/2025

Problem

In lab seven we learned how to smooth real time information collected from the DualShock 4 controller by using moving averages. Raw data collected from sensors, in this lab it was the gyroscope, often change rapidly and have random small fluctuations that make clearly understanding the motion sometime difficult. The goal was to create a C program that calculates the moving averages of the values on all three axis, x, y, and z, of the gyroscope. It reads live controller data from the controller with a pre-provided program called d24rd, it then computes moving averages using a window length provided on input. The program then displays the results, along with the maximum and minimum values for each axis. It keeps doing this until Square button is pressed.

Analysis

The program took the gyroscope axis values as inputs. The main outputs are the smoothed average values, as well as the highest and lowest values stored in the average window. The other outputs were the raw x, y, z values used by the program. The window size is determines by the user's input on the command line. It controls how smooth data will be. Shorter windows, react faster to changes in value but are more sensitive to noise. While larger windows change slower in response to changes in motion, but produce more stable data. The relationship for moving averages can be defines as the average of the last n readings: $\text{Average} = \text{sum of Last } n \text{ Values} / n$. This equations shows this mathematical relationship. This relationship means that when new data comes in, older values are removed and replaced by the latest data read. It uses this to constantly update the moving average.

Design

The program I developed to solve this problem contained arrays and functions to manage and compute the specific data needed for output. First the user is asked to input a window size. It then reads the axis values of the gyroscope continuously, through a scanf in a while loop. The most recent readings are stored in buffers, which are updated with every new line of data. The functions used are avg(), maxmin(), and updatebuffer(). Avg() calculated the averages of values currently in the buffer. Maxmin(), finds the highest and lowest values currently in the buffer. Updatebuffer() shifts the values one spot to the left and adds the newest reading at the end, which is what moves the window to keep thee most recent readings. The main loop, which scans, calculates, and calls the various functions, is ended when the square button is pressed.

Testing

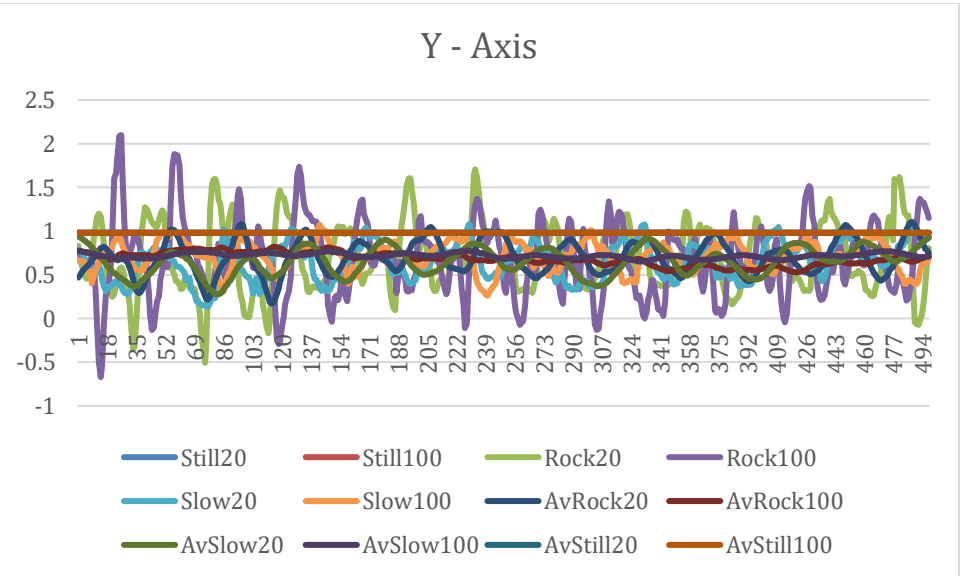
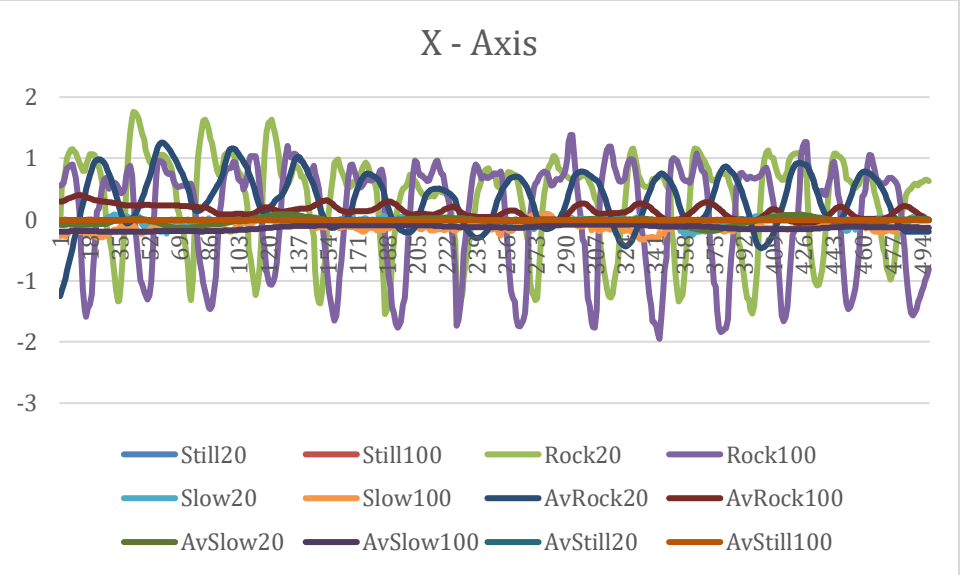
To test the program, I tested three different motion types, still, rocking, and a slow motion of my choosing, what I chose was a back-and-forth tilt. Each motion was then tested using a different window length, 20 and 100. The data was then saved into an Excel file, and I created graphs for each axis containing each motion, to compare the results of the different window lengths and see how they compare. During the still motion, both graphs showed for both window lengths that the line was almost completely flat, showing that there was little motion or noise. While the controller was in the rocking motions, the short window follows the motion close, but appears to be jagged. The long window was significantly smoother, but lagged behind the actual motion. For the slower motion, both windows performed similarly well, but the long window was more stable and easier to read. For quicker motion, I would most likely use a short window length, because keeping up with the motion as data is coming in is important, while the long window length lags behind. But for the slower motion, I would use a longer window length, because both the long and short window perform well, but the stability the long window length provides is an added bonus.

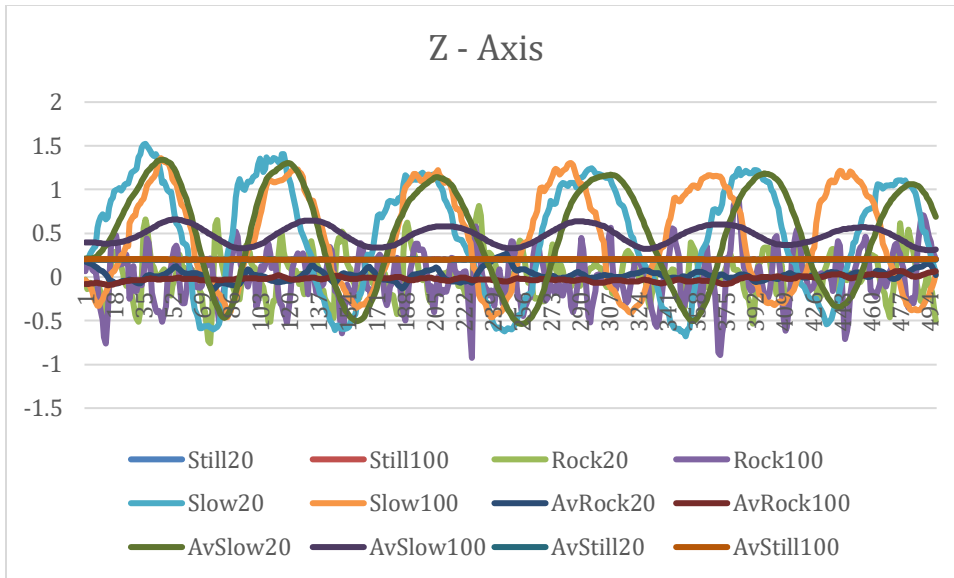
Comments

This lab helped me practice using arrays in a more complex manner, since I had to keep on updating them every loop. Also, it helped me in understanding visually and in practice the difference between using many values to calculate an average as well as using fewer values. It also helped me to integrate loops and functions together, where a loop calls a function, and that function has a loop. The biggest problem that I faced when working on this was an issue on my scanf, I forgot to put parenthesis between my scanf values, which then messed up all the values I received. This helped me to remember that there are small things that can affect your code, and they you need to be careful even with the basics.

Questions and Experiments

1. During the still motion, both graphs showed for both window lengths that the line was almost completely flat, showing that there was little motion or noise. While the controller was in the rocking motions, the short window follows the motion close, but appears to be jagged. The long window was significantly smoother, but lagged behind the actual motion. For the slower motion, both windows performed similarly well, but the long window was more stable and easier to read.





2. For quicker motion, I would most likely use a short window length, because keeping up with the motion as data is coming in is important, while the long window length lags behind. But for the slower motion, I would use a longer window length, because both the long and short window perform well, but the stability the long window length provides is an added bonus.

Implementation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAXPOINTS 10000
```

```
// compute the average of the first num_items of buffer
```

```
double avg(double buffer[], int num_items);
```

```
// update the max and min of the first num_items of array
```

```
void maxmin(double array[], int num_items, double* max, double* min);
```

```
// shift length-1 elements of the buffer to the left and put the new_item on the right
```

```
void updatebuffer(double buffer[], int length, double new_item);
```

```
int main(int argc, char* argv[]) {
```

```
    double x, y, z;
```

```
    int Button_T, Button_C, Button_X, Button_S;
```

```
    int lengthofavg = 0;
```

```
    int count = 0;
```

```
    if (argc > 1) {
```

```
        sscanf(argv[1], "%d", &lengthofavg);
```

```
        printf("You entered a buffer length of %d\n", lengthofavg);
```

```
    } else {
```

```
        printf("Enter a length on the command line\n");
```

```
    return -1;
}
```

```
if (lengthofavg < 1 || lengthofavg > MAXPOINTS) {
    printf("Invalid length\n");
    return -1;
}
```

```
// Buffers for moving average
```

```
double xbuf[lengthofavg];
```

```
double ybuf[lengthofavg];
```

```
double zbuf[lengthofavg];
```

```
while (1) {
```

```
    scanf("%lf, %lf, %lf, %d, %d, %d, %d", &x, &y, &z, &Button_T, &Button_C, &Button_X,
&Button_S);
```

```
    if(Button_S == 1) {
```

```
        break;
```

```
    }
```

```
if (count < lengthofavg) {
```

```
    // fill up buffer until full
```

```
    xbuf[count] = x;
```

```
    ybuf[count] = y;
```

```
    zbuf[count] = z;
```

```

        count++;
    }

    else {

        // shift and add new data point
        updatebuffer(xbuf, lengthofavg, x);
        updatebuffer(ybuf, lengthofavg, y);
        updatebuffer(zbuf, lengthofavg, z);

        // compute averages
        double xav = avg(xbuf, lengthofavg);
        double yav = avg(ybuf, lengthofavg);
        double zav = avg(zbuf, lengthofavg);

        // compute min and max
        double xmax, xmin, ymax, ymin, zmax, zmin;
        maxmin(xbuf, lengthofavg, &xmax, &xmin);
        maxmin(ybuf, lengthofavg, &ymax, &ymin);
        maxmin(zbuf, lengthofavg, &zmax, &zmin);

        // print values as CSV line
        printf("Value(x,y,z): %lf, %lf, %lf, Average(x,y,z): %lf, %lf, %lf\n", x, y, z, xav, yav,
zav);

        fflush(stdout);
    }
}

```

```
    return 0;
}
```

```
double avg(double buffer[], int num_items) {
    double sum = 0;
    for (int i = 0; i < num_items; i++) {
        sum += buffer[i];
    }
    return sum / num_items;
}
```

```
void maxmin(double array[], int num_items, double* max, double* min) {
    for (int i = 1; i < num_items; i++) {
        if (array[i] > *max) *max = array[i];
        if (array[i] < *min) *min = array[i];
    }
}
```

```
void updatebuffer(double buffer[], int length, double new_item) {
    for (int i = 0; i < length - 1; i++) {
        buffer[i] = buffer[i + 1];
    }
    buffer[length - 1] = new_item;
}
```